

CPDAS

Courier and Parcel Delivery Analytics System

Technical Explainer & Setup Guide

Phase 1 explains how the running application works internally: the overall architecture, each of the five modules, how they connect to one another, and where the data actually comes from given that this is explicitly not a create/update/delete application.

Phase 2 is a plain setup guide for getting the project running on a fresh machine: cloning the repository, installing dependencies, and starting the server.

Phase 1

How the System Works

1. How the Entire Website Is Working

CPDAS is a server-rendered Flask application: every page is built on the server and sent to the browser as plain HTML. There is no client-side framework and no JSON API — the only JavaScript in the whole project is `theme.js`, which just toggles a light/dark attribute and remembers the choice in the browser's local storage. The request/response cycle for every single page follows the same shape:

- The browser requests a URL, e.g. `/m1?zone=Z-MIR&date=2026-07-08`.
- `app.py` matches that URL to a route function. Route functions contain no calculations at all — their only job is to read query parameters, open a database connection, call into the relevant module, and hand the result to a template.
- The module function (in `modules/`) runs one or more read-only SQL `SELECT` queries against `cpdas.db`, does the actual arithmetic in plain Python, and returns a dictionary or list of dictionaries.
- The route hands that dictionary to a Jinja2 template (in `templates/`), which fills in a pre-built HTML page and returns it to the browser. Every number is rounded only at this last step, using the filters registered in `app.py` (`percent`, `hours`, `currency`, `similarity_percent`, ...), which all live in `modules/formatting.py`.
- The database connection is closed before the response is sent (see the `try/finally` block in every route).

Every page extends one shared layout, `templates/base.html`: a left sidebar with the five module links, a top bar with the company name and the theme toggle, and a content area that each page fills in. There are two distinct kinds of pages built on top of that layout:

- **Enquiry pages** (`/m1`, `/m2`, `/m3`, `/m4`) — interactive screens where the operations manager picks a filter (a zone and date, a month, a parcel ID) and sees the computed numbers immediately. These live under `templates/forms/`.
- **Reports** (`/reports/r1` through `/reports/r5`) — the five formal, printable documents (R1–R5) named in the specification, styled for clean A4 output via `static/css/print.css`. These live under `templates/reports/`.

All visual styling is driven by CSS custom properties defined in a single file, `static/css/theme.css`. Every other stylesheet only ever references those variables (e.g. `var(--accent)`) and never writes a literal colour itself, so the whole application's colour scheme — light and dark — can be changed by editing that one file.

2. How Each Module Is Working

The five modules named in the specification map directly onto five files under `modules/`. Each one exposes plain functions that take a database connection and some parameters, and return computed results — nothing in these files ever renders HTML or touches Flask.

M1 — Delivery Performance Analytics (modules/m1_delivery.py)

For a given zone and date, this module answers: how many parcels were attempted, how many were delivered, how many failed, what fraction were on time, and what the average delivery time was. The key internal function is `_resolve_daily_outcomes`, which looks at every delivered/failed scan for that zone on that date and resolves each parcel to exactly one outcome: delivered wins if it was both delivered and failed that day, and a parcel with neither scan that day is excluded entirely (it is still in transit). From that resolution, `get_zone_performance` computes the success rate, the on-time rate, and the average delivery time, and also evaluates the “operations review” flag (success below 85% on three consecutive days) and a secondary hub-out-to-delivery timing figure. `get_weekly_zone_performance` re-runs the same resolution across a rolling seven-day window for the R2 report.

M2 — Rider Productivity Analytics (modules/m2_rider.py)

For a given month, this module counts how many parcels each rider delivered and how many delivery attempts they failed, then divides by that rider's duty-day count to get a productivity figure and computes each rider's failed-attempt percentage. `get_all_rider_productivity` also computes the company-wide average productivity across every rider and flags any rider whose productivity is below 70% of that average, or whose fail rate is above 15%, for coaching. `get_failure_reason_breakdown` separately counts how many times each named reason (e.g. “customer unreachable”) appears among a rider's failed attempts.

M3 — COD Reconciliation (modules/m3_cod.py)

For a given rider and date, this module sums the cash-on-delivery amount of every parcel that rider delivered that day (what they collected), looks up what they actually deposited at the hub, and reports the difference. A positive difference is a shortage; a negative one is an over-deposit, usually a counting error. `get_monthly_shortage_rate` aggregates this across a whole month and flags a rider for escalation to the accounts manager if their cumulative shortage rate exceeds 0.5% of everything they collected that month.

M4 — AI Address-to-Zone Matching (modules/m4_matching.py)

This module cleans a parcel's free-text delivery address (lowercasing it, stripping punctuation and bare numbers, and removing generic words like “house” and positional words like “near”) and compares the resulting set of words against each zone's own description using cosine similarity. An optional TF-IDF weighting mode can be turned on instead of the default binary comparison. If the best-matching zone scores at least 80% — and is not tied with a second zone also at or above 80% — the parcel is auto-assigned to it; otherwise it is left for a human dispatcher to sort manually. This is the only module in the entire application that writes anything to the database: every automatic match and every manual override is recorded as a new row in the `zone_assignments` table, and old rows are never changed or removed, which gives a complete history of how a parcel's zone assignment changed over time.

M5 — Reporting (modules/m5_reports.py)

This module does not calculate anything itself. Its five functions (`get_r1_report` through `get_r5_report`) each call into M1–M4, gather the figures those modules already computed, add whatever purely presentational labels a printed report needs (for example, deciding whether a rider's “Note” column should read “Above average” or “Coaching list” based on flags M2 already set), and hand the whole package to a report template.

3. How the Modules Are Working Together

The modules are not independent silos — they are layered, with clear one-way dependencies:

- **modules/text_similarity.py** is a small shared library used by both `modules/m4_matching.py` (at request time, when the app is running) and `scripts/generate_data.py` (once, offline, while building the seed data). Keeping the address-cleaning and cosine-similarity logic in one place guarantees the two callers can never quietly drift apart.
- **modules/formatting.py** is used everywhere a number reaches a screen: `app.py` registers its functions as Jinja2 template filters once, and every single template in the project uses those same filters, so a percentage, an hour figure, or a currency amount is always rounded the same way.
- **M1 depends on M4's output.** This is the most important cross-module link in the system. M1's per-zone figures need to know which zone each parcel belongs to — and that assignment is not stored on the parcel itself, it lives in the `zone_assignments` table that only M4 ever writes to. Concretely, `_resolve_daily_outcomes` in `m1_delivery.py` joins each scan to `zone_assignments` and always takes the row with the highest ID for that parcel (the most recent assignment), so M1's numbers automatically reflect the latest match or dispatcher override M4 has ever recorded for that parcel.
- **M5 depends on all four other modules.** It imports `m1_delivery`, `m2_rider`, `m3_cod` and `m4_matching` and calls straight into them; it is deliberately the only place besides `app.py` that imports more than one of the four analytics modules.
- **app.py imports every module** and is the only place a route ever exists. It is the single point where a URL, a module function, and a template all meet.

One consequence of this layering is worth calling out: because M1's zone figures come from `zone_assignments` rather than from the parcel record itself, a parcel genuinely has no zone at all until M4 has processed it at least once. In the seeded sample data, every historical parcel already has an automatic match recorded (representing matching that would already have happened in the normal course of business before the analytics system ever looked at the data); the handful of demonstration parcels on the M4 screen are deliberately left unmatched so the “Run AI Matching” button has something real to do.

4. Where the Data Is Getting Into the System (It Is Not a CRUD Site)

The specification is explicit that this system must never add, update, or delete a row in any of its five core datasets — parcels, scans, riders, zones, and deposits. Every query the running Flask application makes against those five tables is a plain `SELECT`; there is no form, button, or route anywhere in `app.py` or `modules/` that inserts, updates, or deletes a row in any of them. This is not just a convention — `scripts/verify.py` greps the entire codebase for exactly that pattern and fails loudly if it ever finds one.

So where does the data actually come from? The specification's own framing is that the booking counters, the riders' handheld scanning app, and the cash office already capture this data in the real world, long before CPDAS ever looks at it — CPDAS's job is only to analyse data that already exists somewhere else. This project reproduces that hand-off with two scripts that run **once, offline, before the web server ever starts**:

- `scripts/generate_data.py` deterministically manufactures a realistic dataset (around 8,500 parcels and their scans, eight riders, three zones, and a handful of COD deposits) as plain CSV files under `data/`. It is seeded with a fixed random-number seed, so running it again always produces byte-for-byte identical output. This script plays the role of the booking counters, the handheld app, and the cash office — the upstream systems the specification says already exist.
- `scripts/seed.py` reads those CSV files and loads them into `cpdas.db`, a SQLite database file, using the schema in `schema.sql`. This is the *only* place in the entire project where an `INSERT` statement is allowed to target one of the five protected tables — and it only ever runs from the command line, as part of setup, never while the server is handling a request.

Once `cpdas.db` exists, the Flask application treats it as a fixed, read-only fact base for the rest of its life. The one narrow exception is the `zone_assignments` table, which does not hold business data at all — it holds CPDAS's own computed output (which zone the AI matching, or a human dispatcher, assigned a parcel to). The running application is allowed to `INSERT` new rows there (and only `INSERT` — never update or delete an existing row), which is how M4's "Run AI Matching", batch-assignment, and dispatcher-override actions work without breaking the no-CRUD rule: they add a new fact to an audit trail, they never rewrite history.

The CSV files produced by `generate_data.py` are committed to the git repository (the database file itself is not, since a binary database file in git produces unmergeable conflicts between teammates), so anyone who clones the repository can rebuild the exact same database with one command — `python scripts/seed.py` — without needing to regenerate anything.

Phase 2

Setting Up on a New Machine

1. Cloning the Repository

On the new machine, open a terminal, navigate to wherever the project should live, and run:

```
git clone https://github.com/Omar-Asif/cpdas.git
cd cpdas
```

This downloads the full project, including the committed seed-data CSV files under `data/` and the specification PDF under `spec/`. The database file itself is not part of the repository — it is built locally in the next step.

2. Installing Dependencies

The only requirement is **Python 3.12 or newer**. There is exactly one Python package dependency for the whole project — `Flask` — listed in `requirements.txt`. The project ships a one-shot setup script for each platform that creates an isolated virtual environment, installs that one dependency into it, and builds the database, all in one step.

Windows

`setup.bat`

macOS / Linux

```
chmod +x setup.sh # only needed the first time
./setup.sh
```

Either script does the following, in order:

- Creates a virtual environment in `.venv/` (skipped if one already exists).
- Activates that virtual environment and runs `pip install -r requirements.txt`, which installs `Flask==3.1.*` and nothing else.
- Runs `python scripts/generate_data.py` to produce the CSV files in `data/` (this step is technically redundant on a fresh clone, since those files are already committed, but it is harmless and guarantees a fully reproducible database even if the CSVs were ever deleted).
- Runs `python scripts/seed.py` to build `cpdas.db` from `schema.sql` and those CSV files.

If you would rather run each step by hand instead of using the setup script, the equivalent manual commands are:

```
python -m venv .venv
# Windows:
.venv\Scripts\activate.bat
# macOS / Linux:
source .venv/bin/activate

pip install -r requirements.txt
python scripts/generate_data.py
```

```
python scripts/seed.py
```

3. Running the Website

With the virtual environment already set up (from step 2), starting the server is the same two commands every time afterwards:

Windows

```
.venv\Scripts\activate.bat && python app.py
```

macOS / Linux

```
source .venv/bin/activate && python app.py
```

The terminal will print something like `Running on http://127.0.0.1:5000`. Open that address in any web browser. Leave the terminal window open for as long as the site should stay available — closing it, or pressing `Ctrl+C` inside it, stops the server. The application defaults every page to 8 July 2026, the most recent date the sample dataset covers, so real figures are visible immediately without changing any filters.

To confirm everything installed and seeded correctly before relying on it, two optional checks are available from the same terminal (with the virtual environment still active):

```
python scripts/verify.py          # checks every figure from the spec
python -m unittest tests.test_formulas -v  # unit tests for each formula
```

Both should finish by reporting that every check passed. If either fails after a fresh clone, it almost always means `scripts/seed.py` was not run, or was run before `scripts/generate_data.py` had produced the CSV files.